SVM Classification Using the Iris Dataset

Project Documentation

Platform: Kaggle Notebook | Language: Python | Model: Support Vector Machine (SVM)

Abstract

This project uses a Support Vector Machine classifier to predict Iris flower species from four numeric measurements. The workflow includes dataset inspection, a stratified train/test split, feature standardization, model training, evaluation, a confusion-matrix visualization, and a sample prediction.

1. Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm commonly used for classification. It attempts to find a decision boundary that separates classes with a wide margin. In this notebook, an SVM classifier is trained to identify Iris flower species using four measurements.

2. Project Objectives

Load and inspect the Iris dataset.

Separate input features from the target labels.

Split the data into training and testing sets.

Scale features and train an SVM classifier with an RBF kernel.

Evaluate the model using accuracy, a classification report, and a confusion matrix.

Predict the species for a new flower measurement.

3. Dataset Description

The notebook loads the Iris dataset directly from scikit-learn using load_iris(); no separate CSV upload is required.

The dataset contains 150 samples, four numeric input features, and a target label with three classes. The four features are sepal length, sepal width, petal length, and petal width (all in centimetres).

Target classes: Iris setosa, Iris versicolor, and Iris virginica.

4. Tools and Libraries

Python, NumPy, Pandas, Matplotlib, Seaborn, and scikit-learn.

Key scikit-learn components: train_test_split, StandardScaler, SVC, make_pipeline, accuracy_score, classification_report, and confusion_matrix.

5. Methodology

Step 1 — Import libraries.

Step 2 — Load Iris data and convert the measurements to a Pandas DataFrame.

Step 3 — Inspect the dataset shape, columns, missing values, summary statistics, and target classes.

Step 4 — Define X as the four flower measurements and y as the target class.

Step 5 — Split the data using an 80% training and 20% testing split, random_state=42, and stratify=y.

Step 6 — Build a pipeline that standardizes features with StandardScaler and trains SVC(kernel='rbf', C=1.0, gamma='scale').

Step 7 — Predict labels for the test set and calculate accuracy.

Step 8 — Print precision, recall, and F1-score using classification_report.

Step 9 — Visualize the confusion matrix with Seaborn.

Step 10 — Predict the species for an example flower with measurements [5.1, 3.5, 1.4, 0.2].

6. Model Configuration

Test size: 0.20 (20%); training size: 0.80 (80%).

Random state: 42 for repeatable splitting.

Stratification: preserves the approximate class distribution in train and test sets.

Feature scaling: StandardScaler is applied within a pipeline to avoid fitting the scaler on the test data.

Classifier: SVC with the RBF kernel, C=1.0, and gamma='scale'.

7. Evaluation Metrics

Accuracy: proportion of test examples classified correctly.

Precision: proportion of predicted examples for a class that truly belong to that class.

Recall: proportion of actual examples in a class that were correctly identified.

F1-score: harmonic mean of precision and recall.

Confusion matrix: compares actual labels with predicted labels for each class.

8. Example Prediction

The notebook predicts a class for one example flower with sepal length 5.1 cm, sepal width 3.5 cm, petal length 1.4 cm, and petal width 0.2 cm.

The predicted class is generated when the notebook is run; this documentation does not claim an unexecuted numerical result.

9. How to Run on Kaggle

Open Kaggle and sign in.

Create a new Notebook.

Copy each code cell from the provided svm-datase.ipynb notebook in order.

Run the cells from top to bottom. The Iris dataset is loaded by scikit-learn, so internet access and a separate dataset upload are not required for this example.

Review the displayed dataset information, accuracy, classification report, confusion matrix, and prediction.

10. Conclusion

This project demonstrates a complete beginner-level classification workflow using SVM and the Iris dataset. It covers data loading, inspection, preprocessing, model training, evaluation, visualization, and prediction. For a real project, evaluation should also consider class balance, cross-validation, and performance on data not used during model development.

Appendix A — Notebook Cell Outline

• Cell 1: Import libraries

• Cell 2: Load Iris dataset

• Cell 3: Inspect dataset

• Cell 4: Split features and target

• Cell 5: Create and train SVM pipeline

• Cell 6: Calculate accuracy

• Cell 7: Print classification report

• Cell 8: Plot confusion matrix

• Cell 9: Predict a new flower's species